

Soccer Example

Scott L. Zeger

2025-11-08

Introduction This exercise introduces the use of *Markov Chain Monte Carlo* or *MCMC* to fit mixed effects models to multilevel or longitudinal data. The data are from the 2014 World Cup, where on July 13, 2014 Germany defeated Argentina 1-0 before 74,738 at Maracanã Stadium in Rio de Janeiro, Brazil. There was an estimated global television audience of 1.01 billion viewers for the final game.

Our data are the penalty shots and goals taken during the tournament by 30 of the top players who represent many countries. For each player, we observe their total number of penalty shots attempted and the number they scored. The objective of the analysis is to precisely estimate the penalty shot success rate for each player. It might seem that this is an easy problem since we can just use the fraction scored by each person as the estimate of his unknown success rate. This is the maximum likelihood estimate (mle) under the assumption that each player's rate is a separate parameter of the model. But, should we really estimate as 0.0 the true success rate for the player who took one shot and missed? Do we really think, based upon his data, that he is incapable of ever scoring a penalty shot? Also, the uncertainty in this estimate 0/1 is large. An exact 95% confidence interval obtained by inverting a Fisher-exact test is (0.0,0.975). Is there not a better estimator for which the uncertainty is smaller?

In this context, treating each player as if they are the only one, is an inferior approach. It is much better to acknowledge that the players are from a population of elite professionals and so their unknown scoring rates can usefully be assumed to come from an analogous population of scoring rates. This note presents a Bayesian solution to the estimation problem that addresses the concerns above.

Let's start by looking at the data.

1. For a player who takes one penalty shot and misses, why is the maximum likelihood estimate 0.0 of his true rate undesirable?
2. What might be meant by the expression "borrowing strength" when thinking about how to estimate a particular player's rate?

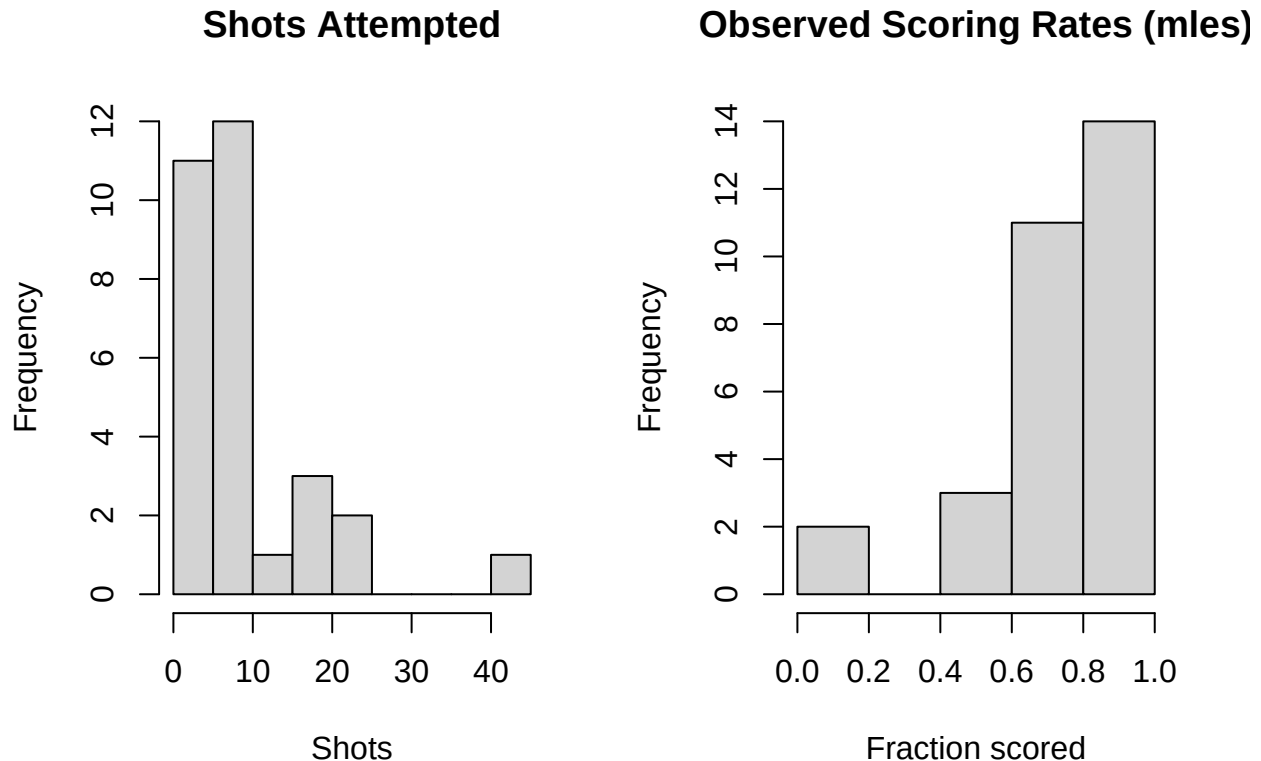
```
setwd("/Users/szeger/Library/CloudStorage/OneDrive-JohnsHopkins/professional/courses/bio650s/bio656.2025")
soccer = read_excel("soccer.xlsx")
d=soccer[,2:10]
rownames(d)=as.vector(soccer[,1])$Name
cols=1:ncol(d)
d[,cols] = apply(d[,cols], 2, function(x) as.numeric(as.character(x)))
d$shots=apply(d[,1],sum)
d$goals=apply(d[,c(3,6,9)],1,sum)
d=d[d$shots>0,]
#
head(d)
```

```
## # A tibble: 6 x 11
##      LM    LS    LG    MM    MS    MG    RM    RS    RG shots goals
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     0     1     0     0     0     0     0     0     0     1     0
## 2     1     2     0     0     1     1     0     1     0     6     1
```

## 3	0	1	3	1	1	0	0	1	2	9	5
## 4	1	3	6	0	2	4	0	1	0	17	10
## 5	0	1	3	0	1	0	0	0	0	5	3
## 6	0	0	0	0	1	0	0	0	2	3	2

The total number of shots taken is recorded in the data frame variable *shots* that ranges from 1 to 41 over the 30 players. For each player, the total shots is stratified by two variables: direction of shot - left (L), middle (M), or right (R) of the goal; and shot outcome - missed (M, columns 1, 4 and 7), saved (S), or scored (G for “Goooooooooalllll”). So *LM* refers to shots to the left side of the goal that were missed. Below, we ignore the direction of the shot and pool misses and saves to focus on each player’s scoring rate without regard to his specific target.

```
# linear approximation Empirical Bayesian
#
# calculate player-specific observed scoring rates - these are the maximum likelihood estimates under t
#
d$p=d$goals/d$shots # calculate score rate
d$p.adjust= ifelse(d$p==0,0.05,ifelse(d$p==1,0.95,d$p))
# calculate asymptotic se and confidence intervals for each players rate
d$se=sqrt(d$p.adjust*(1-d$p.adjust)/d$shots)
ll=NULL;ul=NULL
for(i in 1:nrow(d)) {
  junk=binom.exact(x=d$goals[i],n=d$shots[i])$conf.int
  ll=c(ll,junk[1])
  ul=c(ul,junk[2])
}
d$ll=ll
d$ul=ul
# histogram of the mles
par(mfrow=c(1,2))
hist(d$shots, main="Shots Attempted",xlab="Shots")
hist(d$p, main="Observed Scoring Rates (mles)",xlab="Fraction scored")
```



```
par(mfrow=c(1,1))
```

The histogram of the observed fraction of shots scored (right panel) shows that the estimates are wide ranging. The median estimate is 0.8 with interquartile range (0.675, 0.951).

There are two sources of variation that are combined in this histogram : (1) the heterogeneity among players in their true scoring rates p_i s. We will denote the variance of the p_i s by $\tau^2 = \text{var}(p_i)$; and (2) the statistical errors or differences between $\hat{p}_i$ and p_i . This is measured by the conditional variance $\text{var}(\hat{p}_i|p_i) = p_i(1-p_i)/n_i$ where n_i is the number of shots attempted by player $i = 1, \dots, m$.

3. What is we double the number of shots taken by each player without changing the observed rates (that is, also double the number of scoring shots). What will happen to the two sources of variability among the observed rates?

On the left side of the plot below entitled “Linear approximate empirical Bayes estimates”, the maximum likelihood estimate (mle) $\hat{p}_i$ of each player’s true scoring rate $p_i = y_i/n_i$ is plotted along the black slanted line along with an exact binomial 95% confidence interval obtained by inverting the binomial test of $H_0: p_i = p, p \in (0, 1)$. A 95% confidence interval interval is defined to include all null values not rejected at the $\alpha = 0.05$ level.

What do you notice about the uncertainty in estimates near 0 and 1? They have the widest confidence intervals. Why? Because they are for players who took fewer shots. If we were unsophisticated data analysts, we might make the mistake of ranking players based upon their observed scoring rates. This is a mistake because the best and worst-rated players would be the ones on whom we had the least information.

4. Find a public health or other example, where ranking is done on observed rates so that the highest and lowest ranked units are the ones with the smallest n ’s?

```
# Calculate approximate empirical Bayes estimates of the scoring rates
#
# calculate pbar and se.pbar - scoring rate and standard error across all players
#
pbar=sum(d$goals)/sum(d$shots)
```

```

#
# calculate estimate of total variance among observed rates
#
total.var=var(d$p)
# calculate estimate of variance among true rates as difference between total variance among observed rates and variance among observed rates
#
var.true.rates = max(0,total.var - mean(d$se^2))
var.pbar = var.true.rates/length(d$p) + sum(d$p*(1-d$p)/d$shots)
#
# calculate alpha, the shrinkage fraction from player's observed rate toward the overall average rate
#
d$alpha = var.true.rates/(var.true.rates + d$se^2)
#
# calculate empirical Bayes rate (and its standard error) for each player as compromise between player's observed rate and overall average rate
#
d$p.eb=(1-d$alpha)*pbar + d$alpha*d$p
#
# calculate variance of EB estimate
c1=(d$alpha + (1-d$alpha)*d$shots/sum(d$shots))^2
c2= sum(d$shots^2 * d$total.var.phat)
d$total.var.phat = d$p.adjust*(1-d$p.adjust)/d$shots + var.true.rates
d$se.eb=sqrt( c1*d$total.var.phat + ( (1-d$alpha)/sum(d$shots))^2 * (c2-d$shots^2 * d$total.var.phat) )

par(mfrow=c(1,1))
shakeit=rnorm(length(d$p))*0.01
mle=d$p + shakeit
x.mle=mle*2
eb=d$p.eb + shakeit
x.eb=2*eb+1.5
eb.l=eb-2*d$se.eb
eb.u=eb+2*d$se.eb
mle.l=d$l1
mle.u=d$u1
plot(x.mle,mle,pch="o",xlab="",ylim=c(0,1),xlim=c(-.5,3.6),xaxt="n",ylab="Goal Scoring Rate Estimates",lty=1)
axis(side=2,at=seq(0,1,0.2))
mtext(side=1,"ML",line=1,at=0)
mtext(side=1,"EB",line=1,at=1)
abline(a=0,b=0.5,col="black")
abline(a=-0.75,b=0.5,col="red")
for (j in 1:length(d$p)) {lines(rep(x.mle[j],2),c(mle.l[j],mle.u[j]),col="black")}
points(x.eb,eb,pch="e",col="purple")
for (j in 1:length(d$p)) {lines(rep(x.eb[j],2),c(eb.l[j],eb.u[j]),col="purple")}
for (j in 1:length(d$p)) {lines(c(x.mle[j],x.eb[j]),c(mle[j],eb[j]),col="grey")}

```

Empirical Bayes estimation In the introduction, we asked whether the mle $\hat{p}_i = y_i/n_i$ where y_i is the number of goals out of n_i shots, is a reasonable estimate of p_i . The answer is partly; it is a reasonable summary of the evidence from that single player's results about his true rate, but it is far from optimal if the goal is to predict well over all players. It ignores the fact that these players represent multiple prediction challenges and that their true rates are related to one another, in the sense that they can reasonably be assumed to form a distribution of rates.

To build on the idea that their true rates form a distribution, there exists an off-the-shelf, improved estimate of the p_i s by assuming a 2-level linear mixed effects model, making a simplifying assumption, that the binomial

Linear Approximate Empirical Bayes Estimates

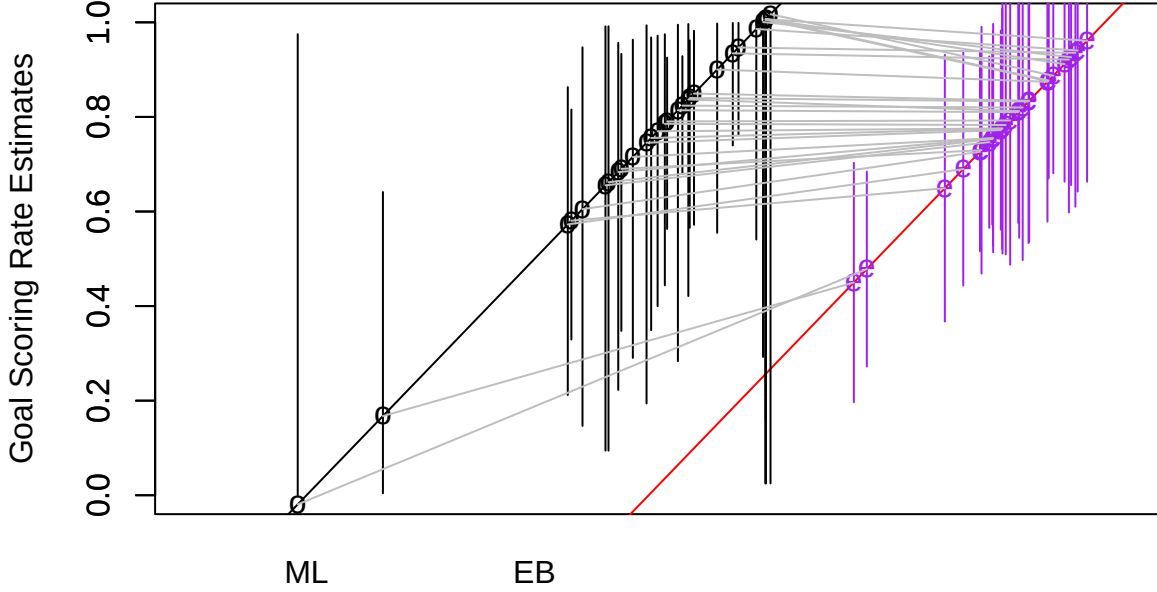


Figure 1: Plot of maximum likelihood (ML-black) and empirical Bayes (EB-purple) estimates (with 95 percent confidence intervals) connected to show differential shrinkage toward the population mean depending on number of shots taken by the player.

distribution can be approximated by the Gaussian distribution with matching mean and variance.

At level-1, we assume the number of goals scored by player i is binomial with true rate p_i and sample size n_i . At level-2, we assume that the true rates p_i follow a Gaussian distribution with mean p_0 and variance τ^2 . If we assume that the binomial distributions are approximately Gaussian with mean p_i and variance $p_i(1 - p_i)/n_i$, then we can use the standard results for the 2-level Gaussian (aka normal) model to obtain the conditional mean of the unknown p_i given the observed rate y_i/n_i : $\tilde{p}_i = E(p_i|y_i) = (1 - \alpha_i)p_0 + \alpha_i y_i/n_i$ where $\alpha_i = \tau^2 / (\tau^2 + p_i(1 - p_i)/n_i)$. Since, we don't know the population mean p_0 and variance τ^2 of the p_i distribution, we estimate them from the data and substitute the estimated values in the formula above. This is the reason for the name "empirical Bayes" estimator.

The estimate for p_0 is just the total number of goals scored by all players divided by their total number of shots $\hat{p}_0 = \sum_i y_i / \sum_i n_i$; the estimate for τ^2 is the amount of excess variance in the observed rates over what you would expect absent heterogeneity in the true rates: $\hat{\tau}^2 = (m - 1)^{-1} \{ \sum_i (y_i/n_i - \hat{p}_0)^2 - \sum_i \hat{p}_i(1 - \hat{p}_i)/n_i \}$. When τ^2 is small, this estimate can be negative, so we use 0 in that case. Another problem is that the estimate $\hat{p}_i(1 - \hat{p}_i)/n_i$ for the binomial variance is 0 when y_i is either 0 or n_i . That fact tends to bias this estimator towards higher values. But, these technical issues will be resolved when we use a more appropriate logistic linear model which we turn to now.

5. For a given n_i , what value for p_i makes the Gaussian approximation of the binomial distribution best?

Let's go back to our 2-level model for the binary indicator whether or not a player makes a shot and derive the exact conditional distribution of p_i given y_i under a more appropriate set of assumptions. We will assume y_i is binomial(p_i, n_i) and $\text{logit}(p_i) = \beta_0 + b_i$ where b_i has a known distribution with mean 0 and variance τ^2 . Then our job is to calculate the expected value and variance of p_i given y_i .

It is possible to write integral equations for the unknown mean and variance above, but until about 1990, it was difficult to evaluate the integrals. An influential paper by Gelfand and Smith solved this problem and revolutionized statistical practice (Gelfand, Alan E., and Adrian FM Smith. "Sampling-based approaches to calculating marginal densities." *Journal of the American statistical association* 85.410 (1990): 398-409.)

The method they introduced to statisticians, called Markov Chain Monte Carlo or MCMC, simulates a sequence of values for all of the unknowns, here model parameters and random effects, from their joint conditional posterior distribution given the data. The sequence follows a Markov chain for which the stationary distribution is the posterior of interest. After a run-in period, the chain gives you as many samples as you want (and for which you can afford to pay computer charges) from any of the marginal or joint distributions of scientific interest, and any other functions of the unknowns that interest you.

In our problem, the main interest is in each player's true scoring rates p_i , for example Lionel Messi's rate. The MCMC will give as many simulated values from the posterior distribution of his true rate given his performance and the model assumptions. His estimate will combine his data with the results for all the other players. Because he shot many times, his observed rate will be the major contributor to his Bayes estimate. In comparison, Diego Forlan (player 1) took only one shot. So his posterior will be more influenced by the average for all players since we have little information in our data about his actual skill.

In this application, the unknown parameters are: (1) β_0 - mean of the random intercepts on the logit scale, and (2) τ^2 - variance of the random intercepts. In Bayesian analysis, we must specify our prior opinion about the unknown parameters and random effects. We will assume that β_0 has a Gaussian prior with variance $1/.44 = 2.27 = 1.5^2$. We chose this prior to put 95% probability between (-3,3) on the logit scale that corresponds to (.05,.95) on the probability scale. This is what Bayesians would call a *weakly-informative* prior in this we are allowing a wide-range of possible mean rates. We are letting the data rather than prior opinion, largely determine the posterior distribution.

The other parameter τ^2 is important to this analysis because it measures the heterogeneity in players' true rates on the logit scale. Again, we want to use a weakly or non-informative prior so the data determine the posterior. How we specify a non-informative prior involves a sequence of steps (lines 167-175 below). The shape of the prior and posterior distributions for this parameter are in the figure below.

Given the prior distributions and the 2-level hierarchical model for the random effects and data, we can implement these assumptions into our own MCMC algorithm using a variety of packages in Rstudio. Two of the most popular are "rjags" and "rstan". Below, we use rjags code that is relatively easy to master given a few examples to mimic like this one. These days rstan is much more popular.

MCMC Output Now it is time to run the MCMC algorithm, done in line 191. The sequence of simulated values is assigned to a dataframe called *sample*. This will include the simulated mean logit rate, across-player variance of the logit rates and the individual random effects for each player. The code calculates the sequence of rates for each player by adding the random effect to the mean rate on the logit scale and transforming it back to the probability scale. So we end up with a sequence of simulated player-specific rates. For each player, the code calculates the mean simulated rate and the "95% credible interval" that includes the middle 95% of simulated values for each parameter and/or random effect.

Displays of results The graphs below compare the Bayes estimates of each player's true scoring rate that borrow information from the whole set of players with the MLEs that use only a player's own shots. The graphs also compare the results of the approximate empirical Bayes results with two fully Bayes solutions. The first one assumes that the random intercepts follow a Gaussian distribution on the logit scale; the second assumes they follow a longer-tailed distribution, the t-distribution with 3 degrees of freedom.

Questions about the results below Here are some questions or tasks for you to address when reviewing and interpreting the results.

6. Interpret the Rjags code in lines 181-219 explaining: (1) the prior specification for the logit-scale fixed effect (intercept) and the variance of the random effects; (2) the assumed distribution for the random effects; and (3) the likelihood of the data given the parameters and random effects.
7. Compare the Rjags code in lines 181-219 with the code in lines 280-314. What is different and what effect does the change in assumption have on the resulting estimates in Figure 6?

8. In a paragraph, explain Figure 1 to a lay reader. What does the figure show? What statistical principle does it illustrate? Offer your best criticism of the ML and Bayes results.
9. Compare the empirical Bayes estimates to the fully Bayes ones in Figures 1 and 2. The comparison is made directly in Figure 3 (or in Figure 6). What are the main differences in the EB vs Bayes estimates; in their confidence (or credible) intervals? What aspects of the methods can help explain these differences?
10. Figure 4 shows the prior and posterior distributions for the standard deviation of the random effect distribution. How does the data change the prior distribution? What is an approximate 95% credible interval for the unknown standard deviation on the logit scale? How would you explain this to a lay person using penalty kick scoring rates?
11. In Figure 7, the posterior distribution for the random effects standard deviation under the t-3 distribution assumption is added to the results in Figure 4. How does the posterior differ between the Gaussian and t-3 models? Why?

```
#
# full Bayesian analysis with Gaussian distribution for random intercepts on the logit scale
ss=d$shots
y=d$goals
x=matrix(rep(1,length(y)),ncol=1)
set.seed(1234321)
#
# model
#
cat("model {
  #
  # set priors
  for (i in 1:J) {
    beta[i] ~ dnorm(0, 0.44)
  }
  log_lambda ~ dunif(-2,4)
  lambda<-exp(log_lambda)
  #
  # observation model specification
  #
  for(i in 1:length(y)) {
    lp[i] <- x[i,] %*% beta
    tp[i] ~ dnorm(lp[i], lambda)
    p[i] <- exp(tp[i])/(1+exp(tp[i]))
    y[i] ~ dbin(p[i], ss[i])
  }
}", file = "FBrates_model.jag")
#
FBrates.data<-list(y=y,ss=ss,x=x,J=ncol(x))
FBrates.params <- c("beta", "lambda","p")
FBrates.inits <- list(beta = rnorm(length(beta)),log_lambda=0, tp=runif(length(y)))
FBrates.model <- jags.model("FBrates_model.jag", data = FBrates.data, inits = FBrates.inits)

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 30
##   Unobserved stochastic nodes: 32
```

```

##      Total graph size: 221
##
## Initializing model
#
# run the MCMC sampler
#
sample <- coda.samples(FBrates.model, var = FBrates.params, n.iter = 10000, n.burnin = 1000, thin = 10)
junk<-summary(sample)
# choose the statistics of interest
#
FBestimate<-data.frame(junk$statistics[3:32,1:2],junk$quantiles[3:32,c(1,5)])
names(FBestimate)<-c("estimate","se","ci.l","ci.u")
post=as.matrix(sample[[1]],ncol=ncol(sample[[1]]))
v.select=c(1:2,4,12,22,32)
work=post[,v.select]
#
# get posterior mean and standard error on the probability (of scoring) scale
pbar=mean(exp(work[,1])/(1+exp(work[,1])))
pbar.se=sqrt(var(exp(work[,1])/(1+exp(work[,1]))))
#
# for comparison, calculate the mle's using each player's data only with approximate confidence interval
ml.est=y/ss
ml.est1=ifelse(ml.est==0,.05,ifelse(ml.est==1,.95,ml.est))
ml.se=sqrt(ml.est1*(1-ml.est1)/ss)
ml.ci.l=NULL
ml.ci.u=NULL
for (j in 1:length(y)) {
  junk=binom.test(x=y[j],n=ss[j],p=0.5)
  ml.ci.l=c(ml.ci.l,junk$conf.int[1])
  ml.ci.u=c(ml.ci.u,junk$conf.int[2])
}
Mleestimate=cbind(ml.est,ml.se,ml.ci.l,ml.ci.u)
saveit=list(d,Mleestimate,FBestimate,sample)

shakeit=rnorm(nrow(FBestimate))*0.01
mle=Mleestimate[,1]+shakeit
x.mle=mle*2
fb=FBestimate[,1]+shakeit
x.fb=2*fb+1.5
plot(x.mle,mle,pch="o",xlab="",ylim=c(0,1),xlim=c(-.5,3.6),xaxt="n",ylab="Estimated Goal Scoring Rate",
axis(side=2,at=seq(0,1,0.2))
mtext(side=1,"ML",line=1,at=0)
mtext(side=1,"Bayes-G",line=1,at=1)
abline(a=0,b=0.5,col="black")
abline(a=-0.75,b=0.5,col="red")
for (j in 1:nrow(Mleestimate)) {lines(rep(x.mle[j],2),c(Mleestimate[j,3],Mleestimate[j,4]))}
points(x.fb,fb,pch="b",col="green")
for (j in 1:nrow(FBestimate)) {lines(rep(x.fb[j],2),c(FBestimate[j,3],FBestimate[j,4]),col="green")}}
for (j in 1:nrow(FBestimate)) {lines(c(x.mle[j],x.fb[j]),c(mle[j],fb[j]),col="grey")}}

plot(FBestimate[,1],d$p.eb,ylab="Linear Appox to EB Estimates",xlab="Bayes Estimate",xlim=c(0.0,1.0),yl
abline(0,1)
for (j in 1:length(d$p)) {
  lines(rep(FBestimate[j,1],2),c(d$p.eb[j]-2*d$se.eb[j],d$p.eb[j]+2*d$se.eb[j]),col="green")
}

```


Full Bayes Estimation with Gaussian Distribution

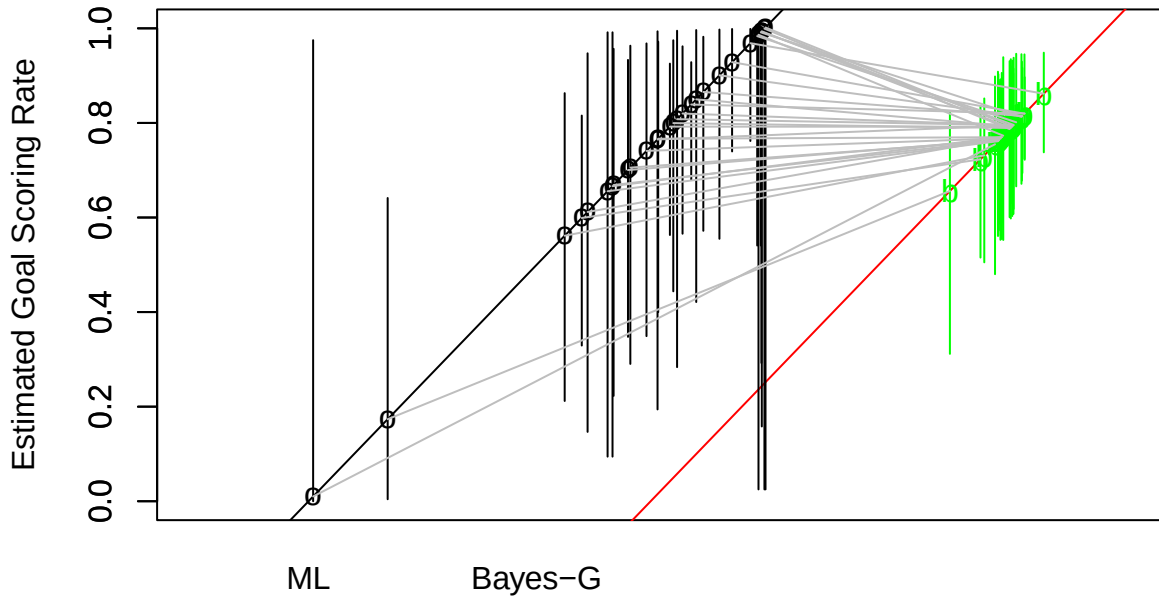


Figure 2: Plot of maximum likelihood (ML-black) and full Bayes estimates (Bayes-G- green) assuming a Gaussian distribution of logit rates. (Bayes-G- green). Estimates are connected to show differential shrinkage toward the population mean depending on number of shots taken by the player.

```
lines(c(FBestimate[j,3],FBestimate[j,4]),rep(d$p.eb[j],2),col="purple")
}
```

```
prior.lambda=exp(runif(10000,min=-2,max=4))
prior.sigma=1/sqrt(1/prior.lambda)
work.g=saveit[[4]][[1]][,"lambda"]
plot(density(sqrt(1/work.g)),xlab="Std Dev of logit(p[i])",xlim=c(0,3),main="Prior vs Posterior for SD")
lines(density(prior.sigma),col="red")
text(x=c(0.8,2.0),y=c(1.1,.3),label=c("Posterior","Prior"),col=c("green","red"))
```

```
ss=d$shots
y=d$goals
x=matrix(rep(1,length(y)),ncol=1)
set.seed(1234321)
#
# model
#
cat("model {
#
# set priors
  for (i in 1:J) {
    beta[i] ~ dnorm(0, 0.44)
  }
  log_lambda ~ dunif(-2,4)
  lambda<-exp(log_lambda)
#
# model specification
```

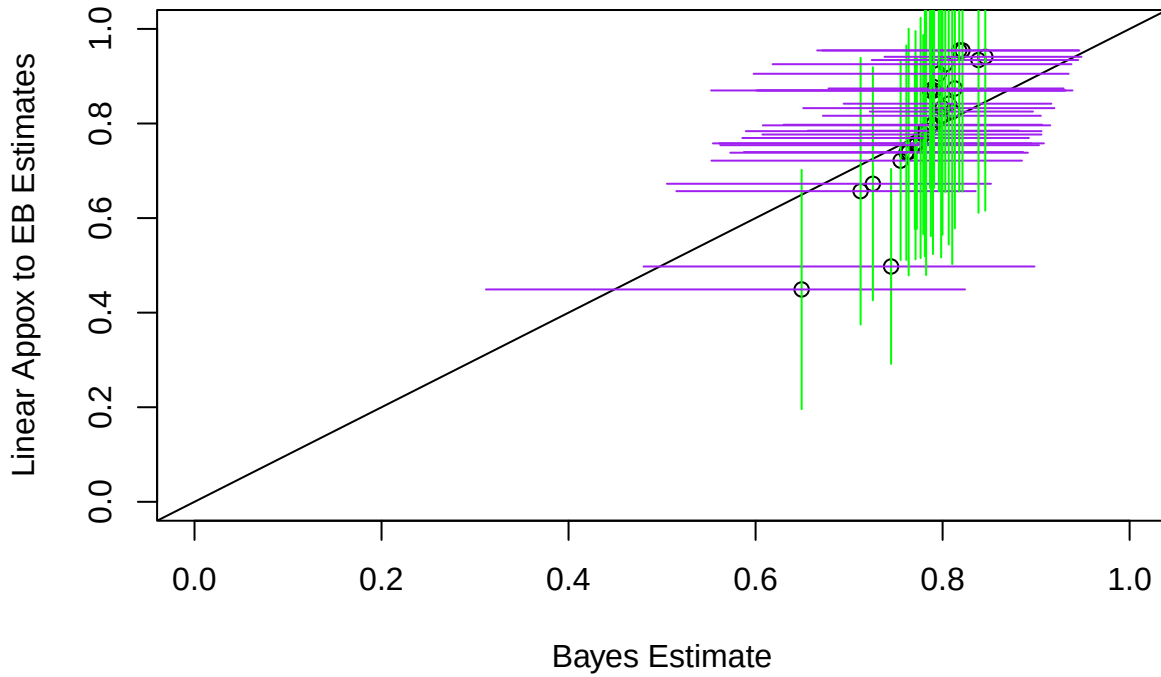


Figure 3: Comparison of approximate EB estimates with confidence intervals (green) versus full Bayes estimates with credible intervals (purple) under the Gaussian model for random intercepts.

Prior vs Posterior for SD of $\text{logit}(p[i])$ with Gaussian Distribution

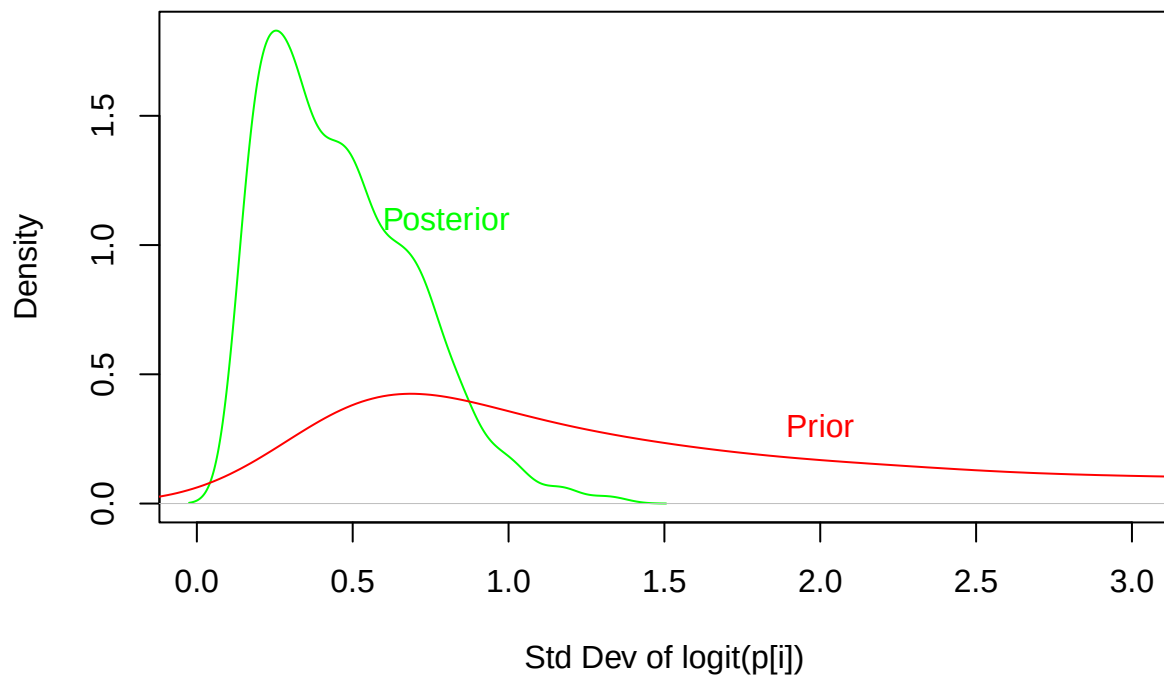


Figure 4: Prior and posterior distributions for the standard deviation of the player random effect from the fully Bayes model with Gaussian random intercept on the logit scale.

```

#
  for(i in 1:length(y)) {
    lp[i] <- x[i,] %*% beta
    tp[i] ~ dt(lp[i], lambda,3)
    p[i] <- exp(tp[i])/(1+exp(tp[i]))
    y[i] ~ dbin(p[i], ss[i])
  }
  }", file = "FBrates_model.jag")
#
FBrates.data.t<-list(y=y,ss=ss,x=x,J=ncol(x))
FBrates.params.t <- c("beta", "lambda","p")
FBrates.inits.t <- list(beta = rnorm(length(beta)),log_lambda=0, tp=runif(length(y)))
FBrates.model.t <- jags.model("FBrates_model.jag", data = FBrates.data.t, inits = FBrates.inits.t)

## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 30
##   Unobserved stochastic nodes: 32
##   Total graph size: 222
##
## Initializing model

#
samples.t <- coda.samples(FBrates.model.t, var = FBrates.params.t, n.iter = 10000, n.burnin = 1000, thin=1)
junk.t<-summary(samples.t)
p.rows=(length(beta)+2):(length(beta)+1+length(y))
FBestimate.t<-data.frame(junk.t$statistics[p.rows,1:2],junk.t$quantiles[p.rows,c(1,5)])
colnames(FBestimate.t)<-c("eb.est","eb.se","eb.ci.l","eb.ci.u")
post.t=as.matrix(samples.t[[1]],ncol=ncol(samples.t[[1]]))
v.select=c(1:2,4,12,22,32)
work.t=post[,v.select]

pbar.t=mean(exp(work.t[,1])/(1+exp(work.t[,1])))
pbar.se.t=sqrt(var(exp(work.t[,1])/(1+exp(work.t[,1]))))

saveit.t=list(d,Mleestimate,FBestimate.t,samples.t)

shakeit=rnorm(nrow(FBestimate.t))*0.01
mle=Mleestimate[,1]+shakeit
x.mle=mle*2
fb.t=FBestimate.t[,1]+shakeit
x.fb.t=2*fb.t+1.5
plot(x.mle,mle,pch="o",xlab="",ylim=c(0,1),xlim=c(-.5,3.6),xaxt="n",ylab="Estimated Goal Scoring Rate",
axis(side=2,at=seq(0,1,0.2))
mtext(side=1,"ML",line=1,at=0)
mtext(side=1,"EB",line=1,at=1)
abline(a=0,b=0.5,col="black")
abline(a=-0.75,b=0.5,col="red")
for (j in 1:nrow(FBestimate.t)) {lines(rep(x.mle[j],2),c(Mleestimate[j,3],Mleestimate[j,4]))}
points(x.fb.t,fb.t,pch="e",col="red")
for (j in 1:nrow(FBestimate.t)) {lines(rep(x.fb.t[j],2),c(FBestimate.t[j,3],FBestimate.t[j,4]),col="blue")}
for (j in 1:nrow(FBestimate.t)) {lines(c(x.mle[j],x.fb.t[j]),c(mle[j],fb.t[j]),col="grey")}
```

Full EB Estimation with t Distribution

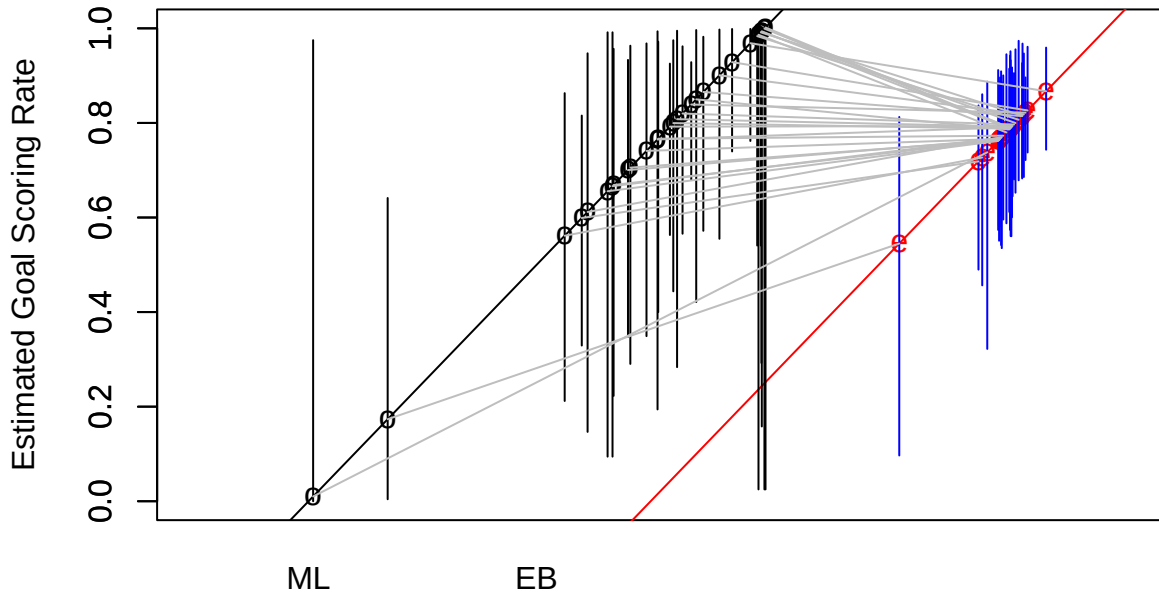


Figure 5: Plot of estimates with confidence intervals for: (1) maximum likelihood (m-black); (2) empirical Bayes (e-purple); (3) full Bayes with Gaussian random effects (g- green); and full Bayes with t-3 random effects (t - blue).

```
# #
# redo plot above with only full Bayes solutions to compare them more easily
#
names(d)

## [1] "LM"          "LS"          "LG"          "MM"
## [5] "MS"          "MG"          "RM"          "RS"
## [9] "RG"          "shots"       "goals"       "p"
## [13] "p.adjust"    "se"          "ll"          "ul"
## [17] "alpha"       "p.eb"        "total.var.phat" "se.eb"

delta=0.15
plot(1:30,d$p,pch="m",col="black",xlab="",xlim=c(0,32),ylab="Estimates with 95% CI",ylim=c(0,1))
points(1:30+delta,d$p.eb,col="purple",pch="e")
points(1:30+2*delta,FBestimate[,1],col="green",pch="g")
points(1:30+3*delta,FBestimate.t[,1],col="blue",pch="t")
for(j in 1:30) {
  lines(rep(j,2),c(d$ll[j],d$ul[j]),col="black")
  lines(rep(j,2)+delta,c(d$p.eb[j]-2*d$se.eb[j],d$p.eb[j]+2*d$se.eb[j]),col="purple")
  lines(rep(j,2)+2*delta,c(FBestimate[j,3],FBestimate[j,4]),col="green")
  lines(rep(j,2)+3*delta,c(FBestimate.t[j,3],FBestimate.t[j,4]),col="blue")
}

prior.lambda=exp(runif(10000,min=-2,max=4))
prior.sigma=1/sqrt(1/prior.lambda)
work.t=saveit.t[[4]][[1]][,"lambda"]
plot(density(sqrt(1/work.t)),xlab="Std Dev of logit(p[i])",xlim=c(0,3),col="blue")
lines(density(prior.sigma),col="red")
```

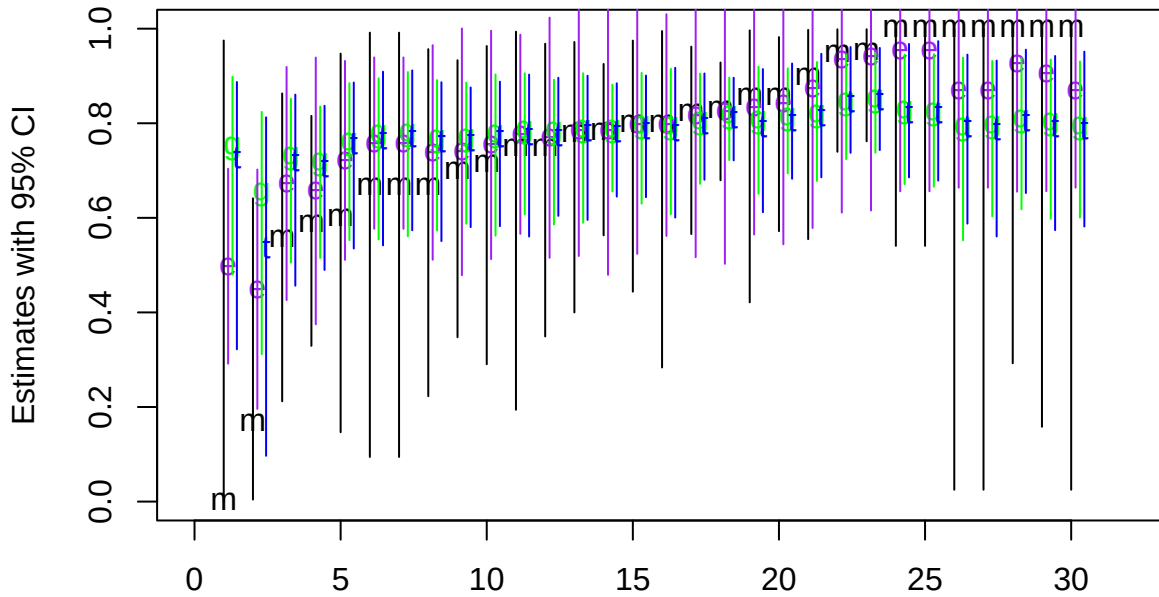


Figure 6: Plot of maximum likelihood (ML-black), empirical Bayes (e - purple), full Bayes estimates with Gaussian random effects (g-green), t distribution with 3 degrees of freedom of the logit rates. (t- blue). Estimates are connected to show differential shrinkage toward the population mean depending on number of shots taken by the player.

```
lines(density(sqrt(1/work.g)),col="green")
text(x=c(0.8,0.5, 2.0),y=c(1.0,2.0,.3),label=c("Posterior-G","Posterior-t3","Prior"),col=c("green","blue","black"))

write.csv(cbind(MLestimate,FBestimate,FBestimate.t),file="soccer.example.results.csv")
save(saveit,file='soccer.results.gaussian.Rdata')
save(saveit.t,file='soccer.results.t.Rdata')
```

What about direction of shot? Thus far, we have addressed questions about the population average and each player's rate of scoring. We have ignored the direction in which the shot was taken. One of the most daring and potentially embarrassing shots is toward the middle of the goal. If the goalie does not move left or right as the shot is being taken, a shot down the middle is a sure save - an embarrassment to the shooter. If the goalie does dive toward the edge of the goal, the shot down the middle is a sure winner and raises the counterfactual question: why didn't the goalie just stand still.

Let's see if shots down the middle of the goal are more or less successful than those toward either side. We will introduce a covariate $x_{ij} = 1$ if shot j by player i is down the middle and 0 otherwise. We further assume the logistic linear mixed model

$$\log\left(\frac{p_{ij}}{1-p_{ij}}\right) = \beta_{0i} + x_{ij}\beta_1$$

where β_{0i} is assumed to be a Gaussian variable with mean β_0 and variance τ^2 . This random intercept represents the variation among players in their scoring rate when shooting to the sides. The regression coefficient β_1 is the log odds of scoring down the middle versus to the sides, assumed here to be constant across players. If the data are sufficient, we might also allow β_1 to be random indicating some players do better down the middle while others score more often toward the sides.

```
setwd("/Users/szeger/Library/CloudStorage/OneDrive-JohnsHopkins/professional/courses/bio650s/bio656.2021")
soccer = read_excel("soccer.xlsx")
#
# create dataframes: (1)
```

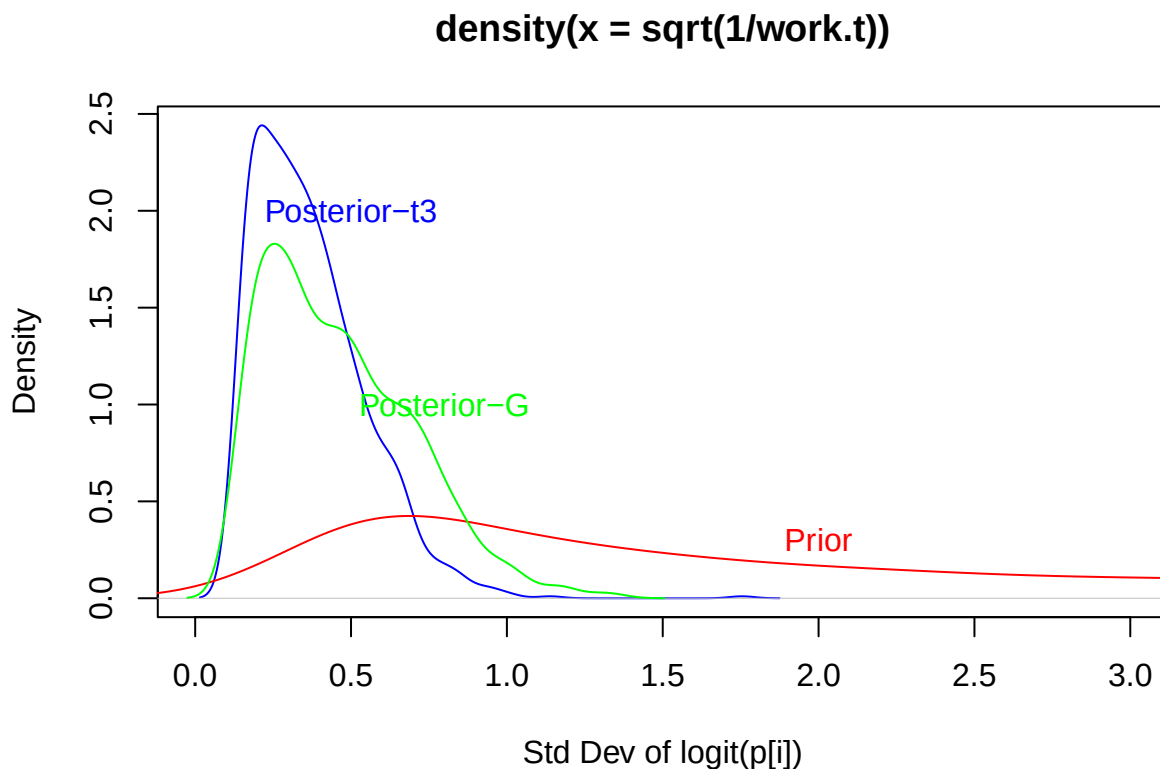


Figure 7: Prior and posterior distributions for the standard deviation of the random effects on the logit scale from the Gaussian and t-3 distributions.

```
#
d = soccer %>%
  tidyr::gather(key = "colname", value = "value", -Name) %>%
  dplyr::arrange(Name)
d = d %>%
  dplyr::mutate(
    side = substr(colname, 1, 1),
    outcome = substr(colname, 2, 2),
    outcome = case_when(
      outcome == "M" ~ "missed",
      outcome == "G" ~ "goal",
      outcome == "S" ~ "saved",
      TRUE ~ outcome
    ),
    side = case_when(
      side %in% c("L", "R") ~ "side",
      side %in% "M" ~ "middle",
      TRUE ~ side
    )
  )
d = d %>%
  group_by(Name, side, outcome) %>%
  summarise(
    value = sum(value)
  )
```

```

d = d %>%
  group_by(Name, side) %>%
  mutate(shots = sum(value)) %>%
  filter(outcome == "goal") %>%
  select(-outcome) %>%
  rename(goals = value) %>%
  relocate(goals, .after = last_col())

shot_both = d %>%
  group_by(Name) %>%
  filter(!any(shots == 0))
#
d1 = d %>% filter(shots > 0)
d1 = d1 %>% mutate(middle=ifelse(side=="middle",1,0))
#
# run MCMC to fit regression model with random player intercept
#
# set up outcome and design matrices
#
y=d1$goals
x=cbind(rep(1,length(d1$goals)),d1$middle)
n=d1$shots
#
J=ncol(x)
set.seed(1234321)
beta=rep(0,J)
#
# model
#
cat("model {
  #
  # set priors
  for (i in 1:J) {
    beta[i] ~ dnorm(0, 0.44)
  }
  log_lambda ~ dunif(-2,4)
  lambda<-exp(log_lambda)
  #
  # model specification
  #
  for(i in 1:length(y)) {
    lp[i] <- x[i,] %*% beta
    tp[i] ~ dt(lp[i], lambda,3)
    p[i] <- exp(tp[i])/(1+exp(tp[i]))
    y[i] ~ dbin(p[i], n[i])
  }
}", file = "FBrates_model.jag")
#
FBrates.data.t<-list(y=y,n=n,x=x,J=ncol(x))
FBrates.params.t <- c("beta", "lambda","p")
FBrates.inits.t <- list(beta = rnorm(length(beta)),log_lambda=0, tp=runif(length(y)))
FBrates.model.t <- jags.model("FBrates_model.jag", data = FBrates.data.t, inits = FBrates.inits.t)

## Compiling model graph

```

```

##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 47
##   Unobserved stochastic nodes: 50
##   Total graph size: 438
##
## Initializing model
#
samples.t <- coda.samples(FBrates.model.t, var = FBrates.params.t, n.iter = 10000, n.burnin = 1000, thin = 1)
junk.t<-summary(samples.t)
p.rows=(length(beta)+2):(length(beta)+1+length(y))
FBestimate.t<-data.frame(junk.t$statistics[p.rows,1:2],junk.t$quantiles[p.rows,c(1,5)])
colnames(FBestimate.t)<-c("eb.est","eb.se","eb.ci.l","eb.ci.u")
post.t=as.matrix(samples.t[[1]],ncol=ncol(samples.t[[1]]))
v.select=c(1:2,4,12,22,32)
work.t=post[,v.select]

pbar.t=mean(exp(work.t[,1])/(1+exp(work.t[,1])))
pbar.se.t=sqrt(var(exp(work.t[,1])/(1+exp(work.t[,1]))))

saveit.t=list(d,Mleestimate,FBestimate.t,samples.t)

```